

QA Test Report

Wapi / Flying-Tea-Squad

Code & Architecture Review | Backend Go/Gin/MongoDB + Frontend React Native/Expo

Date: 2025-07-15 | Prepared by: QA Analysis | Status: **NOT READY FOR PRODUCTION**

1. Executive Summary

This report documents the findings of a static code & architecture review of the Wapi / Flying-Tea-Squad repository. The platform provides user authentication (email/password with bcrypt plus Google and Facebook OAuth), a Livepeer-based streaming service, and a custodial wallet that integrates M-PESA (Safaricom Daraja API) and Solana (FLOWER token mint/burn/transfer). The mobile frontend is built with Expo / React Native and TypeScript.

The codebase has a solid structural foundation: clear handler/service/model separation, Swagger documentation, and a meaningful test suite for the M-PESA service and auth flows. **However, the application is not ready for production:** the review identified **5 Critical**, **8 High**, and **8 Medium** issues.

2. Severity Summary

Severity	Count	Key Areas
Critical	5	Auth security, refresh-token flow, token forgery
High	8	Wallet integrity, authorization (IDOR), CORS, credential defaults
Medium	8	Config bugs, dead code, frontend hygiene
Low / Quality	7	Logging, error handling, missing rate limiting

3. Critical Findings - Must Fix Before Release

C1. Frontend/Backend Refresh-Token Contract Mismatch (App-Breaking)

Description: The backend `POST /refresh` returns `{ "access_token": ..., "refresh_token": ... }`, but the frontend interceptor reads `res.data.tokens.access / res.data.tokens.refresh` (which is undefined). Every automatic token refresh fails, force-logging-out the user on the first 401.

Location: `wapi/backend/internal/handlers/auth.go` vs `wapi-frontend/utils/axiosconfig.ts`

C2. Hardcoded Default JWT Secret

Description: `GetJWTSecret()` falls back to the string `your-secret-key` when `JWT_SECRET` is unset. Any attacker can forge valid tokens for any user.

Location: `wapi/backend/internal/config/config.go`

C3. OAuth State Never Validated (CSRF / Login Hijacking)

Description: Both `/auth/google` and `/auth/facebook` generate a random state value, but the callback handlers never verify it, enabling login CSRF / session fixation.

Location: `wapi/backend/internal/handlers/auth.go`

C4. Missing Authorization on Stream Endpoints (IDOR)

Description: `GET /api/v1/streams/:id`, `DELETE /api/v1/streams/:id`, and `PUT /:id/viewers` have no ownership check against the JWT `user_id`. Any authenticated user can view/delete others' streams and manipulate viewer counts.

Location: `wapi/backend/internal/routes/routes.go` + stream handlers

C5. JWT Signing Algorithm Not Enforced

Description: The JWT middleware never verifies `token.Method` is `HS256`, leaving the service open to algorithm-confusion attacks (e.g. `alg: none`).

Location: `wapi/backend/internal/middleware/auth.go`

4. High Findings - Fix Before Release

H1. CORS Wildcard Origin + Credentials (Invalid Combination)

Description: AllowOrigins = ["*", ...] combined with AllowCredentials = true is rejected by browsers.

Location: wapi/backend/internal/middleware/cors.go

H2. Wallet Callback Not Idempotent - Double Credit / Double Mint

Description: Every STK callback credits the wallet and mints FLOWER. Daraja retries callbacks, causing duplicate credit and duplicate on-chain mint.

Location: wapi/backend/internal/handlers/wallet.go,
internal/services/wallet/service.go

H3. Gifting to Non-Existent Users + Funds Atomicity Gap

Description: ensureWallet creates a wallet for any ObjectID. If crediting the recipient fails after a successful Solana transfer, the sender balance is not restored and the Solana tx is not reversed.

Location: internal/services/wallet/service.go

H4. Sensitive KeyReference Exposed in Wallet Response

Description: GET /wallet/balance returns the custodial key_reference field to the client.

Location: wapi/backend/internal/models/wallet.go

H5. Hardcoded Default Livepeer API Key

Description: The Livepeer client falls back to default-api-key when LIVEPEER_API_KEY is unset.

Location: wapi/backend/modules/stream/livepeer/livepeer_client.go

H6. M-PESA Callback IP Allowlist Broken Behind Proxy

Description: isTrustedSafaricomSource uses c.ClientIP(); behind a reverse proxy / ngrok the source IP is the proxy, so legitimate callbacks receive 403.

Location: wapi/backend/internal/handlers/wallet.go

H7. No Upper Bounds on Amount Fields

Description: kes_amount and flower_amount are unbounded int64 with no phone-format validation server-side.

Location: wapi/backend/internal/handlers/wallet.go

H8. Signup Accepts Empty / Weak Passwords Server-Side

Description: AuthRequest has no binding:"required" and no minimum-length enforcement (validation only exists in the frontend).

Location: wapi/backend/internal/models/user.go

5. Medium Findings - Fix Recommended

ID	Finding	Location
M1	GetSolanaSignerMode() has unreachable code returning a Facebook redirect URL after the real return	internal/config/config.go
M2	GetMpesaResultURL / GetMpesaTimeoutURL fall back to the callback URL (wrong semantics)	internal/config/config.go
M3	Livepeer HTTP client has no timeout - requests can hang forever	modules/stream/livepeer/livepeer_client.go
M4	loginStore.resetForm() references a 'success' field that does not exist in the type	store/loginStore.tsx
M5	console.log(resp.user) leaks the full user object to the console	app/(auth)/login.tsx
M6	register.tsx sends confirm_password to backend (ignored, but unnecessary)	app/(auth)/register.tsx
M7	useAxios creates a new axios instance every render - stale token closures	utils/axiosconfig.ts
M8	Wallet.MpesaPhone never persisted despite the field existing	internal/services/wallet/service.go

6. Low / Code Quality Findings

Generic error masking - many handlers return "Server error" / "Invalid request", hiding root cause.

GetStream returns 500 for a missing stream instead of 404 (mongo.ErrNoDocuments is not mapped).

Livepeer CreateStream uses unbounded context.Background().

No rate limiting on login / signup / wallet endpoints (brute-force and abuse risk).

No structured logging - uses log.Printf; no request IDs.

Duplicate token logic - refresh/generate code exists in both handler and middleware; consider a shared auth service.

No frontend test framework - zero unit/component tests.

7. Test Coverage Assessment

Existing tests (positive signal for QA):

Test file	What it covers	Needs live MongoDB?
cmd/main_test.go	Smoke test only - does not test the real router	Yes
services/mpesa/service_test.go	Strong - STK push, B2C, reversals, C2B, token caching (mocked Daraja)	No
services/solana/service_test.go	Thin - mint payload + burn validation only	No
services/wallet/service_test.go	ensureWallet helper only	Yes
models/mpesa_test.go	STK callback payload parsing (envelope + flat)	No
test/authss/	Strong integration - signup/login/refresh/logout/OAuth/protected routes	Yes

Coverage gaps - recommend regression tests:

Stream handlers (create/get/end/list) - zero coverage

Stream endpoint authorization (IDOR) - zero coverage

CORS and JWT signing-method middleware - zero coverage

Wallet buy/withdraw/transfer/callback flows - not tested

Callback idempotency / replay - not tested

The refresh-token contract (C1) - not tested (this is why the bug slipped through)

Frontend - no Jest/Testing Library tests at all

8. Recommended QA Test Plan

Priority 1 - Regression tests for Critical issues:

Priority 2 - Wallet integrity:

Priority 3 - Config & robustness:

- P1.1 - Token refresh flow - verify actual JSON keys match frontend expectations (C1)
 - P1.2 - Forge JWT with default secret and alg:none - both must be rejected (C2, C5)
 - P1.3 - OAuth callback with wrong / missing state - must be rejected (C3)
 - P1.4 - User A tries to GET / DELETE User B's stream - must be 403/404 (C4)

 - P2.1 - Replay the same STK callback twice - balance must credit only once (H2)
 - P2.2 - Gift FLOWER to a non-existent user - must be rejected (H3)
 - P2.3 - Check wallet response does NOT include key_reference (H4)

 - P3.1 - Run without JWT_SECRET and LIVEPEER_API_KEY - the app must fail fast, not use defaults (C2, H5)
 - P3.2 - Verify CORS preflight with credentials works in a real browser (H1)
 - P3.3 - Verify M-PESA callbacks arrive through ngrok / proxy (IP allowlist behavior) (H6)
-

9. Verdict

Quality gate: FAIL - do not release until Critical items (C1-C5) are resolved.

The codebase has a solid structural foundation with a meaningful M-PESA and auth test suite. However, the refresh-token contract mismatch breaks the core app flow, and the security issues (default JWT secret, missing OAuth state validation, missing stream authorization, no signing-algorithm check) are exploitable. These findings should be assigned to the responsible development teams. QA should track them and add the regression tests outlined in Section 8 once fixes land.